



IMSE 785

Apache Spark

Spark SQL

Shing Chang
IMSE, K-State

Running Spark to Replace Hadoop Ecosystem

- Structured Data Analysis
 - Spark SQL for Hive SQL
 - Select syntax: <https://docs.databricks.com/spark/latest/spark-sql/language-manual/select.html>
- Interactive SQL
 - Spark SQL for Impala
 - Using Databricks Community Cloud
- Machine Learning Analysis
 - MLlib for Mahout
- Real-time Streaming Data Analysis
 - Spark streaming for Storm

Intro to Databricks

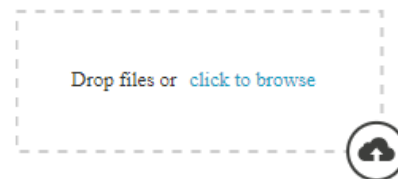
- The founders (Matei Zaharia) in UC Berkeley AMPLab created a company called Databricks to commercialize Apache Spark
- Spark 1.0 released in 2014; Spark 2.0 released in 2016; Spark 3.0 released in 2019 Spark is written in Scala and runs on Java Virtual Machine (JVM)
- Spark can be used from Python, Java, Scala, R, or SQL.
- Three options to practice Spark: downloading and installing Apache Spark on your own PC or laptop, using an IMSE VM server, or running a web-based version in **Databricks Community Edition**
- This tutorial is to setup the second option using a SQL example
 - <https://community.cloud.databricks.com/login.html>

Welcome to databricks™



Explore the Quickstart Tutorial

Spin up a cluster, run queries on preloaded data, and display results in 5 minutes.



Import & Explore Data

Quickly import data, preview its schema, create a table, and query it in a notebook.



Create a Blank Notebook

Create a notebook to start querying, visualizing, and modeling your data.

Common Tasks

-  New Notebook
-  Create Table
-  New Cluster
-  New Job
-  New MLflow Experiment New
-  Import Library
-  Read Documentation

Recents

-  Quickstart Notebook (1)
-  Quickstart Notebook

What's new in v3.3

- Instance Pools
- [Databricks Runtime 6.0 will drop Python 2 support](#)

[View latest release notes](#)

Create a **Cluster** named Quickstart

Create a quickstart cluster

1. In the sidebar, right-click the **Clusters** button and open the link in a new window.
2. On the Clusters page, click **Create Cluster**.
3. Name the cluster **Quickstart**.
4. In the Databricks Runtime Version drop-down, select **5.4 (Scala 2.11, Spark 2.4.3)**.
5. Click **Create Cluster**.

Create Cluster

New Cluster | Cancel Create Cluster

Cluster Name

QS 

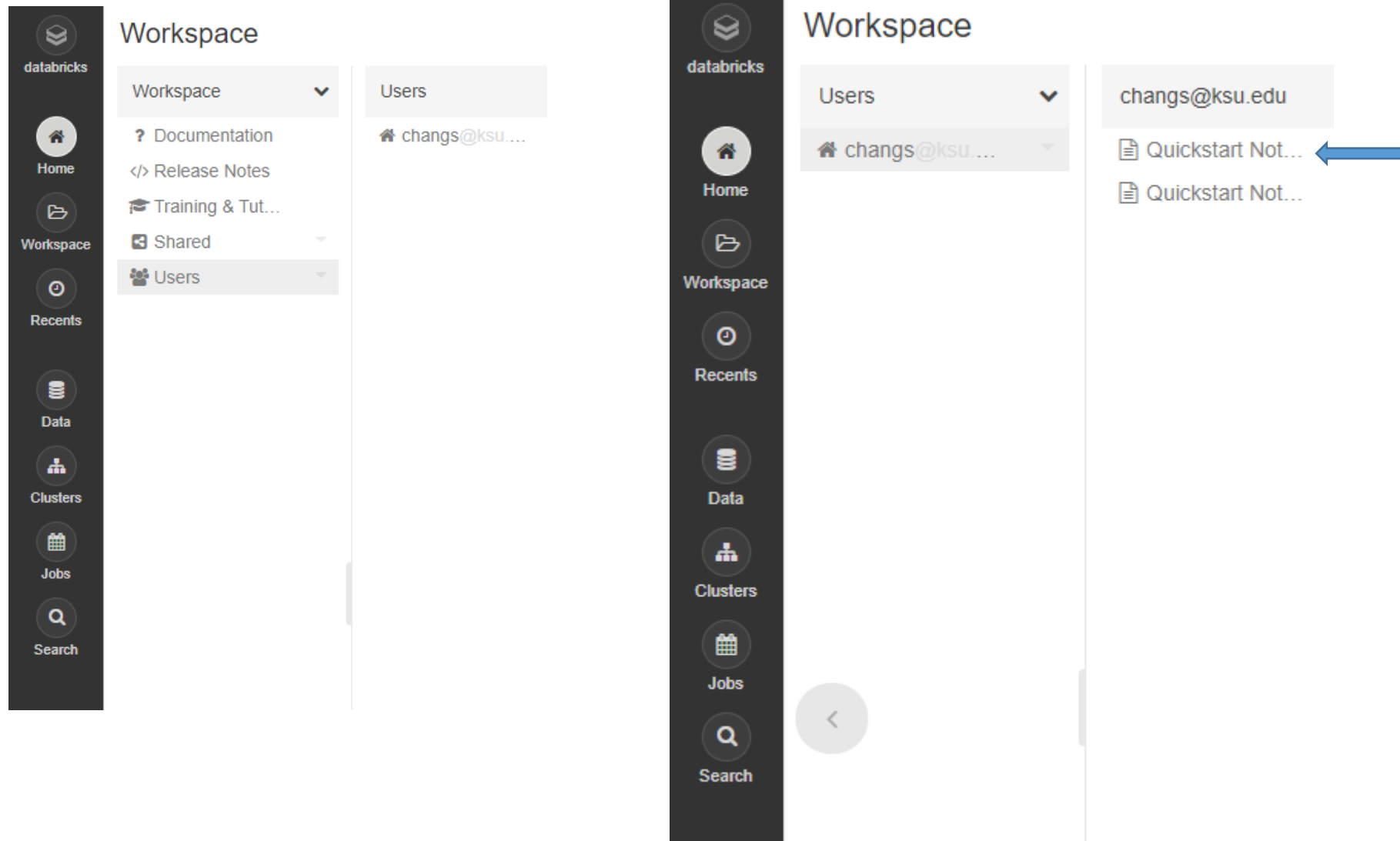
Cluster Mode 

Standard | 

Databricks Runtime Version  [Learn more](#)

Runtime: 5.4 (Scala 2.11, Spark 2.4.3) | 

Databricks Community Edition (Workspace)



Databricks **Quickstart Notebook** Tutorials

- Once you have created a cluster called Quickstart, you can run a Notebook containing a tutorial (in 5 minutes), which demonstrate both the SQL and the use of %python to run SQL

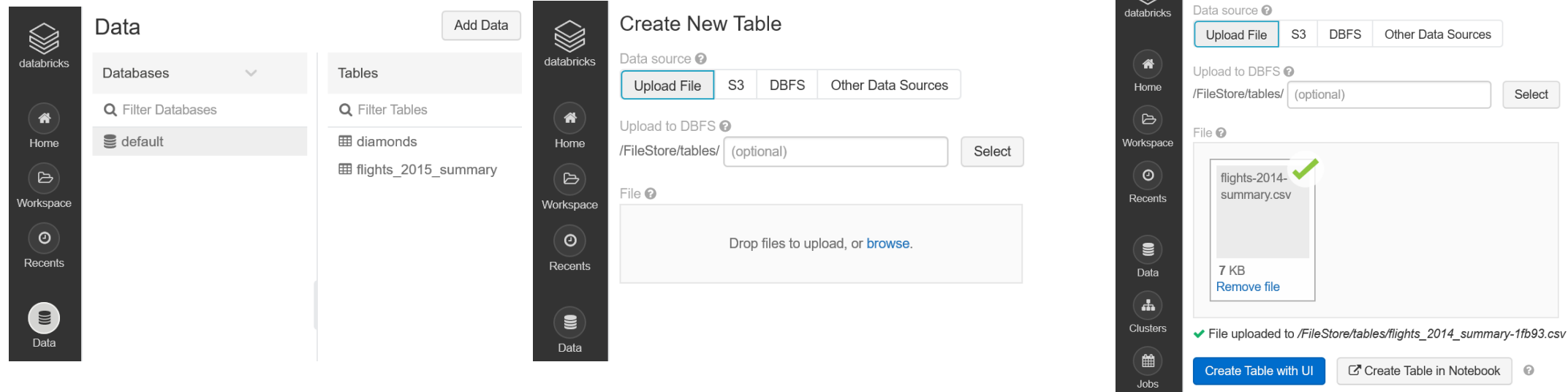
Cmd 3

Attach the notebook to the cluster and run all commands in the notebook

1. Return to this notebook.
2. In the notebook menu bar, select  Detached ▾ > **Quickstart**.
3. When the cluster changes from  to , click  **Run All**.

DBFS – Databricks file system

- <https://docs.databricks.com/data/databricks-file-system.html>
 - The Databricks cloud's default is /FileStore/tables/table_name
 - The upload of any external file is to DBFS
-
- /FileStore: Imported data files, generated plots, and uploaded libraries. See [FileStore](#).
 - /databricks-datasets: Sample [public datasets](#).
 - /databricks-results: Files generated by downloading the [full results](#) of a query.
 - /databricks/init: Global and cluster-named (deprecated) [init scripts](#).
 - /user/hive/warehouse: Data and metadata for non-external Hive tables.



Upload a file and
read it in a df
(dataframe)

- Step 1. click on data on the left panel, and click on Add Data
- Step 2. Drop a file (e.g. flights-2014.summary.csv) into the middle panel saying “Drop files to uoload,...”
- Step 3. Click on “Create Table in Notebook”, then follow the notes.

SQL command creates a table from a Databricks dataset

Cmd 5

```
1 DROP TABLE IF EXISTS diamonds;  
2  
3 CREATE TABLE diamonds  
4 USING csv  
5 OPTIONS (path "/databricks-datasets/Rdatasets/data-001/csv/ggplot2/diamonds.csv", header "true")  
6
```

Cmd 6

```
1 SELECT * from diamonds
```

Selects color and price columns, averages the price, and groups and orders by color

Cmd 11

```
1 SELECT color, avg(price) AS price FROM diamonds GROUP BY color ORDER BY color
```

Convert the table to a chart

Under the table, click the bar chart  icon.

Using Python to read the csv file

Cmd 7



```
1 %python
2 diamonds = spark.read.csv("/databricks-datasets/Rdatasets/data-001/csv/ggplot2/diamonds.csv", header="true", inferSchema="true")
3 diamonds.write.format("delta").save("/delta/diamonds")
```

Cmd 8

```
1 DROP TABLE IF EXISTS diamonds;
2
3 CREATE TABLE diamonds USING DELTA LOCATION '/delta/diamonds/'
```

Cmd 9

```
1 SELECT * from diamonds
```

The next command manipulates the data and displays the results

cmd 17

```
1 %python
2 from pyspark.sql.functions import avg
3
4 display(diamonds.select("color","price").groupBy("color").agg(avg("price")).sort("color"))
```

Shift+Enter to run [shortcuts](#)

Reference

- Chapter 4 of Bengfort and Kim (2016), Data Analytics with Hadoop – An Introduction for Data Scientists, Orilly.
- Spark Tutorial for Beginners
<https://www.youtube.com/watch?v=QaoJNXW6SQo>
- SQL Syntax
 - <https://docs.databricks.com/spark/latest/spark-sql/language-manual/select.html>
- Spark Examples
- <https://spark.apache.org/examples.html>

Select

```
SELECT [hints, ...] [ALL|DISTINCT] named_expression[, named_expression, ...]  
  FROM relation[, relation, ...]  
    [lateral_view[, lateral_view, ...]]  
  [WHERE boolean_expression]  
  [aggregation [HAVING boolean_expression]]  
  [ORDER BY sort_expressions]  
  [CLUSTER BY expressions]  
  [DISTRIBUTE BY expressions]  
  [SORT BY sort_expressions]  
  [WINDOW named_window[, WINDOW named_window, ...]]  
  [LIMIT num_rows]
```

named_expression:

```
: expression [AS alias]
```

relation:

```
| join_relation  
| (table_name|query|relation) [sample] [AS alias]  
: VALUES (expressions)[, (expressions), ...]  
  [AS (column_name[, column_name, ...])]
```

expressions:

```
: expression[, expression, ...]
```

sort_expressions:

```
: expression [ASC|DESC][, expression [ASC|DESC], ...]
```